



AIT | ACADEMY OF INFORMATION TECHNOLOGY

Python for AI & Data Science — Part 2

Day 3: Working with Data Using Python

Master data manipulation: Strings, list operations, JSON structures, filtering, comprehensions, and preparing data for NumPy & Pandas.

 Day 3

 1.5 – 2 Hours

What Did We Learn in Day 2?

✓ Core Python Foundations

✓ Variables

✓ Data Types

✓ Lists

✓ Dictionaries

✓ If / Elif / Else

✓ For Loops

✓ Functions

✓ Import / Libraries

🎯 Remember the Core Goal

Python is not the final goal. It is merely our vehicle to navigate and engineer Artificial Intelligence.

We learn syntax only to empower data pipelines, clean datasets, and feed modern machine learning models.



Pathway: Python → Data Science → Machine Learning →
Generative AI

Day 2 to Day 3 Evolution

Day 2: How Python Works

Variables & Data Types

Lists & Dictionaries

Conditions & Loops

Reusable Functions

Day 3: How Python Works with DATA

Create & Ingest Data

Filter & Clean Records

Transform & Comprehensions

Aggregate & Prepare for AI

Understanding Strings in Python

TEXT REPRESENTATION

```
name = "Aisha"  
city = "Aurangabad"  
course = "Artificial Intelligence"
```

```
# Strings hold alphanumeric text  
print (type (name)) #
```

Strings in Artificial Intelligence

Text is the foundation of Natural Language Processing (NLP) and Large Language Models (LLMs):

 User Prompts

 Customer Reviews

 Email Spam Filter

 PDF Documents

 Social Feeds

 Chatbot Inputs

String Indexing & Methods

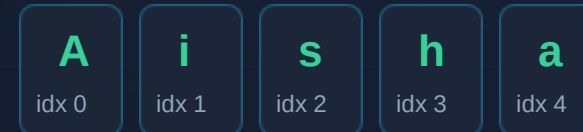
STRING METHODS

```
name = "Aisha"
```

```
print (name[0])      # 'A'
print (len (name))   # 5
print (name.upper ()) # "AISHA"
print (name.lower ()) # "aisha"
```

Zero-Based Indexing

Python strings are ordered sequences of characters starting at index 0:



 `len(name)` returns the total character count including spaces.

Cleaning Messy Text for AI

Raw real-world text is messy:

name = " Aisha " → name.strip() removes whitespace.

Core methods: upper(), lower(), strip(), replace().

The AI Rule: Raw Text → Cleaning → Tokenization → Machine Learning Model.

Application of Text Processing



Information
Retrieval

ntal
s

Qu
Ans

Working with Lists

LIST STRUCTURE

```
marks = [75, 82, 91, 68, 88]
```

Accessing elements

```
print (marks[0]) # Output: 75
```

```
print (marks[2]) # Output: 91
```

Last element

```
print (marks[-1]) # Output: 88
```

☰ Why Lists Matter

A list is a mutable, ordered sequence capable of storing collections of numerical features, experimental trials, or sensor measurements.



Indices start at **0**. Negative indices count backward from the end!

Adding Elements to a List

+ 1. `append()` — Add at End

Inserts an item at the very end of the list.

```
marks = [75, 82, 91]
marks.append(95)

print (marks)
# [75, 82, 91, 95]
```

→ 2. `insert()` — Specific Position

Inserts value at designated index (pos, val).

```
marks = [75, 82, 91]
marks.insert (1, 80)

print (marks)
# [75, 80, 82, 91]
```

⚡ `append()` operates in $O(1)$ constant time, making it ideal for streaming data ingest.

Removing and Sorting Data



remove()

Removes the first occurrence of a specific value.

```
marks.remove(68)
```



sort()

Sorts list elements in ascending order in-place.

```
marks.sort()
```



reverse()

Reverses the order of the list elements in-place.

```
marks.reverse()
```



These methods are essential when structuring and organizing training batches for AI models.

List Slicing Essentials

SLICE SYNTAX: [START : STOP]

```
marks = [75, 82, 91, 68, 88]
```

```
# First 3 values (index 0, 1, 2)
```

```
print (marks[:3])    # [75, 82, 91]
```

```
# From position 2 to end
```

```
print (marks[2:])    # [91, 68, 88]
```

```
# Last 2 values
```

```
print (marks[-2:])   # [68, 88]
```

✂ Slicing = Selecting Subsets

Slicing allows you to extract precise chunks of data without altering the original list.

In Machine Learning, slicing is the fundamental mechanism used to split features X from labels y and partition training sets from test sets.

📌 list[start:end] includes start up to but excluding end.

Real-World Student Dataset

DATASET.PY

```
students = [  
    {"name": "Aisha" , "age" : 21, "city" : "Aurangabad" , "marks" :  
85},  
    {"name": "Rahul" , "age" : 22, "city" : "Mumbai" , "marks" : 72},  
    {"name": "Sara" , "age" : 20, "city" : "Pune" , "marks" : 91},  
    {"name": "Omar" , "age" : 21, "city" : "Aurangabad" , "marks" :  
65},  
    {"name": "Zoya" , "age" : 22, "city" : "Mumbai" , "marks" : 78}  
]
```

Our Practical Benchmark

We will use this exact dataset throughout today's session to filter, transform, summarize, and prepare data.

Notice how this structure combines **strings** (categorical features) and **integers** (numerical features).



This is the universal standard for JSON responses from AI APIs.

Understanding Data Structures

Name (Categorical)	Age (Discrete)	City (Categorical)	Marks (Continuous Feature)
"Aisha"	21	Aurangabad	85
"Rahul"	22	Mumbai	72
"Sara"	20	Pune	91
"Omar"	21	Aurangabad	65
"Zoya"	22	Mumbai	78

Python Representation: Outer List [] contains individual Dictionary rows { }. Each dictionary represents one complete record.

MENTAL MODEL List of Dictionaries as Tables

🔗 Python Data Structure

```
students = [  
    {"name": "Aisha" , "marks" : 85},  
    {"name": "Rahul" , "marks" : 72},  
    {"name": "Sara" , "marks" : 91}  
]
```

🗄 Database & Pandas Mapping

- 🗄 Outer List → Full Table / DataFrame
- ≡ Dictionary → Single Row / Record
- 🔑 Dictionary Key → Column Name / Feature
- ✅ Dictionary Value → Cell Value / Feature Value

💡 Mastering this mental model makes learning **Pandas DataFrames** intuitive and effortless.

Accessing Nested Records

TWO-STEP KEY ACCESS

```
# Step 1: Access first student record
print (students[0])
# {'name': 'Aisha', 'age': 21, ...}

# Step 2: Access specific key inside student
print (students[0]["name" ]) # Aisha
print (students[0]["marks" ]) # 85
```

Double Bracket Indexing

students[0] retrieves the dictionary at index 0.

["name"] then looks up the value associated with that key inside the retrieved dictionary.



This exact double-indexing syntax applies to multi-dimensional tensors and nested APIs.

Iterating Over Datasets

ITERATING OVER RECORDS

```
for student in students:  
    print (student["name" ])
```

OUTPUT

Aisha

Rahul

Sara

Omar

Zoya

Why Loops?

Without loops, you would have to write manual lines for every single item: `print(students[0])`, `print(students[1])`...

With a loop, the exact same 2 lines of code process 5 students, 5,000 students, or 5,000,000 records automatically.

Extracting Multiple Attributes

EXTRACTING FIELDS

```
for student in students:  
    print (student["name"], student["marks"])
```

OUTPUT

```
Aisha 85  
Rahul 72  
Sara 91  
Omar 65  
Zoya 78
```

🎯 The Big Takeaway

Loop = Process multiple records automatically.

You extract multiple features in parallel, paving the way for feature transformation and data pipelines.



This is automated data ingestion in action.

What is Data Filtering?

FILTER: MARKS > 80

```
for student in students:
    if student["marks"] > 80:
        print (student["name"])
```

FILTERED OUTPUT:

Aisha

Sara

Selective Filtering

Pairing for with an if statement checks every record and retains only those satisfying our criteria.

In AI workflows, filtering removes noisy data, extracts high-confidence samples, and creates specialized cohorts.

Filtering Records by Condition

FILTER: CITY == 'AURANGABAD'

```
for student in students:
    if student["city"] == "Aurangabad":
        print (student["name"])
```

OUTPUT

Aisha
Omar

Real-World Filtering Examples

-  Filter customers located in Mumbai
-  Filter students who passed examinations
-  Filter e-commerce products above ₹1,000
-  Filter employees from the IT department
-  Detect financial transactions above ₹50,000

Building Filtered Datasets

PRESERVING FILTERED DATA

```
top_students = []

for student in students:
    if student["marks"] > 80:
        top_students.append(student)

print (top_students)
```

Store, Don't Just Print

Printing outputs to console is only for debugging. Real AI pipelines collect filtered objects into a new data structure.

Original Dataset → Filter Condition → New Cleaned Dataset

`top_students` can now be safely passed to training models or visualization routines.

Intro to List Comprehension

Traditional For-Loop

```
passed = []  
  
for mark in marks:  
    if mark ≥ 60:  
        passed.append(mark)
```

Verbose, requires 4 lines and manual append.

List Comprehension

```
passed = [  
    mark for mark in marks  
    if mark ≥ 60  
]
```

Identical output: Elegant, faster, and concise.

 **Formula:** [expression for item in iterable if condition]

Transforming Data with Lists

MATHEMATICAL TRANSFORMATION

```
marks = [50, 60, 70, 80]
```

```
# Double every mark in a single expression
```

```
double_marks = [mark * 2 for mark in marks]
```

```
print (double_marks)
```

```
# Output: [100, 120, 140, 160]
```

Feature Engineering

List comprehension allows you to apply mathematical operations across an entire sequence simultaneously.

Common AI transformations include normalizing values between 0 and 1, converting temperatures, and scaling pixel intensities.

Comprehension for Filtering

ONE-LINE RECORD FILTER

```
top_students = [  
    student  
    for student in students  
    if student["marks"] ≥ 80  
]  
  
for s in top_students:  
    print (s["name"]) # Aisha, Sara
```

Ubiquitous in AI Codebases

You will see list comprehensions across every major AI framework:

- ✓ Tokenizing sentences into lower-case words
- ✓ Discarding empty or corrupted records
- ✓ Extracting labels: [s['marks'] for s in students]
- ✓ Preparing feature matrices

Reusable Code with Functions

FUNCTION DEFINITION

```
def calculate_average (marks):  
    return su (marks) / len (marks)  
        m  
  
# Using our custom function:  
marks = [85, 72, 91, 65, 78]  
avg = calculate_average (marks)  
  
print (avg) # Output: 78.2
```

DRY: Don't Repeat Yourself

If you calculate average marks for 10 classrooms, you should never copy-paste the formula 10 times.

Wrap logic into a function. Update the function once, and the entire AI data pipeline updates automatically.

✓ Encapsulation ensures zero duplicate errors across datasets.

Useful Built-in Functions

Function	Code Example	Output for [85, 72, 91, 65, 78]	Data Science Purpose
len()	len(marks)	5	Count samples, compute batch size
sum()	sum(marks)	391	Aggregate loss, compute totals
max()	max(marks)	91	Peak values, maximum likelihood
min()	min(marks)	65	Baseline tracking, minimum threshold
sorted()	sorted(marks)	[65, 72, 78, 85, 91]	Rank ordering, quantile discovery

 These built-in functions run fast compiled C routines underneath standard Python.

Statistical Summaries

QUICK STATISTICAL SUMMARY

```
marks = [85, 72, 91, 65, 78]
```

```
print ("Total:" , su (marks))
print ("Highest:" m ma (marks))
print ("Lowest:" , min (marks))
print ("Average:" , su (marks) / len (marks))
m
```

391

TOTAL MARKS

91

HIGHEST MARK

65

LOWEST MARK

78.2

AVERAGE MARK

Anonymous Lambda Functions

</> Standard Function

```
def square(x):  
    return x * x  
  
print (square(5)) # 25
```

⚡ Inline Lambda

```
square = lambda x: x * x  
  
print (square(5)) # 25
```

💡 A **lambda** is a concise, one-line anonymous function used when creating a full def is unnecessary.

Custom Sorting with Lambda

SORTING DICTIONARIES

```
# Sort students by their marks field
students.sort (
    key=lambda s: s["marks" ]
)

for s in students:
    print (s["name" ], s["marks" ])
```

```
Omar 65 → Rahul 72 → Zoya 78 → Aisha 85 → Sara 91
```

↓ Key Extractors in AI

Real-world datasets constantly require custom sorting criteria:

- 📊 Ranking prediction confidence scores
- 🕒 Sorting sensor logs chronologically
- 🏆 Selecting Top-K results in recommendation engines

Handling Nested Data

NESTED DICTIONARY

```
student = {  
    "name" : "Aisha" ,  
    "marks" : {  
        "python" : 85,  
        "sql" : 90,  
        "ai" : 88  
    }  
}  
  
# Accessing nested subject marks  
print (student["marks"] ["python" ]) # 85
```

Multi-Dimensional Properties

Real-world data is rarely flat. An entity contains sub-entities (e.g., student contains subject marks).

Navigating nested dictionaries using chained brackets [key1][key2] is identical to parsing raw JSON responses from cloud services.

Hierarchical Data Models

Data Hierarchy Tree

```
Student (Object)
├── name: "Aisha"
└── marks (Sub-Dictionary)
    ├── python: 85
    ├── sql: 90
    └── ai: 88
```

Bridge to Modern Web APIs

REST APIs and cloud databases format their payloads in trees identical to this.

When calling OpenAI's ChatGPT API or Gemini API, the response is delivered as nested dictionaries. This leads directly to **JSON**!

JSON & Python Dictionaries

JSON (JavaScript Object Notation)

```
{  
  "name" : "Aisha" ,  
  "marks" : 85  
}
```

Universal text format for data exchange across the web.

Python Dictionary

```
student = {  
    "name" : "Aisha" ,  
    "marks" : 85  
}
```

Native Python data structure with 1-to-1 syntax compatibility.

 Because JSON looks like Python dictionaries, Python can serialize and deserialize API data instantly.

API to AI Processing Pipeline

The Modern AI Architecture:

App / User → REST API → JSON Payload → Python Ingestion → AI Model → Inference Result.

When you build custom agents, Python is the orchestration glue parsing JSON tokens into actionable features.

LAYER OF THE MODERN SOC PL

